Worksheet 10: Making an R Package

Making an R package

Making an R package is relatively easy, and in today's worksheet we will learn how to make and and install an R package locally. First, we will need the following packages.

```
install.packages(c("devtools", "roxygen2", "testthat", "knitr", "usethis"))
```

Structure of an R package

A simple R package folder must have the following structure:

1. There is a folder called `R` that contains all the functions that a user will use and any other functions needed.
2. There is a file called `Description` that contains the description of the R package.
3. There is a file called `Namespace` that tells R what other package this depends on this, and what other dependencies does it have.
4. Finally, you will need a folder that contains all documentations.

A package that is very useful for this is the `roxygen2` package. But first, let's create a folder called `myPackage` and then create a folder called `R` inside it. Inside the `R` folder, add a script file (you can name it anything) and then paste the following code in it:

```
#' Add two numbers, or two vector of numbers
#'
#' @param x1 A number or a vector of numbers
#' @param x2 A number or a vector of numbers of the same length as x1
#' @return A number of a vector of numbers which is the sum of the inputs
#' @export
#' @examples
#' add_numbers(1,2)
#' add_numbers(c(1,4), c(5,6))
add_numbers <- function(x1, x2)
{
  rtn <- x1 + x2
  return(rtn)
}
```

Next, we add a `DESCRIPTION` text file in the folder. Please go through the details here: https://r-pkgs.org/description.html.

Once a description file is ready, set your working directory right before the `myPackage` folder and then run the command

```
roxygenize("myPackage/")
```

This will create your R package! Go check the folder and see that a Documentation folder `man` has been added to the folder.

Checking and Installing an R package

You can check whether a package will install correctly by using the `devtools` package. Install this package and run

```
library(devtools)

# check Package
check("myPackage")

# build the package
build("myPackage")
```

If the checks pass, you can build the package. This made a `tar.gz` file locally. This is your package!

```
# install
install.packages("myPackage_0.0.1.tar.gz", repos = NULL, type="source")
```

Your package is installed! Now you can load the package and add numbers!

```
library(myPackage)
add_numbers(5,6)
```

```
[1] 11
```

Run `?add_numbers` to see the help page.

Exercises

1. Make my `GetColorPalette` package! Since you will need other functions in R you will need to import them in the documentation part of the function.